

```

import os
import traceback
import requests
from datetime import datetime
from flask import Flask, jsonify, request
from flask_cors import CORS
import yfinance as yf

app = Flask(__name__)
CORS(app)

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN", "")
CHAT_ID = os.environ.get("CHAT_ID", "")
DASHBOARD_URL = os.environ.get("DASHBOARD_URL", "https://sree789p.github.io/nifty")

CACHE = {}
CACHE_TTL = 300

TICKERS = {}
TICKERS["nifty"] = "^NSEI"
TICKERS["banknifty"] = "^NSEBANK"
TICKERS["nasdaq"] = "^IXIC"
TICKERS["sp500"] = "^GSPC"
TICKERS["nikkei"] = "^N225"
TICKERS["hangseng"] = "^HSI"
TICKERS["usd_inr"] = "INR=X"
TICKERS["crude"] = "CL=F"
TICKERS["gold"] = "GC=F"
TICKERS["vix"] = "^VIX"
TICKERS["india_vix"] = "^INDIAVIX"
def fetch_data(symbol):
    try:
        t = yf.Ticker(symbol)
        hist = t.history(period="30d", interval="1d")
        if hist.empty:
            return {"price": None, "change_pct": None, "rsi": 50.0, "above_sma20":
            close = hist["Close"].dropna()
            price = round(float(close.iloc[-1]), 2)
            change_pct = None
            if len(close) >= 2:
                prev = float(close.iloc[-2])
                change_pct = round((price - prev) / prev * 100, 2)
            rsi = 50.0
            if len(close) >= 14:

```

```

        delta = close.diff()
        gain = delta.clip(lower=0).tail(14).mean()
        loss = -delta.clip(upper=0).tail(14).mean()
        if loss and loss > 1e-6:
            rs = gain / loss
            rsi = round(float(100 - (100 / (1 + rs))), 1)
        else:
            rsi = 100.0
    above_sma20 = 0
    if len(close) >= 20:
        sma20 = float(close.tail(20).mean())
        above_sma20 = 1 if price and price > sma20 else 0
    change_1w = None
    if len(close) >= 6:
        prev_week = float(close.iloc[-6])
        change_1w = round((price - prev_week) / prev_week * 100, 2)
    return {"price": price, "change_pct": change_pct, "change_1w": change_1w,
except Exception as e:
    print("fetch error " + symbol + ": " + str(e))
    return {"price": None, "change_pct": None, "rsi": 50.0, "above_sma20": 0,

def get_cached(symbol):
    now = datetime.now().timestamp()
    if symbol in CACHE:
        data, ts = CACHE[symbol]
        if now - ts < CACHE_TTL:
            return data
    data = fetch_data(symbol)
    CACHE[symbol] = (data, now)
    return data

def compute_features(market):
    def chg(key):
        return market.get(key, {}).get("change_pct") or 0.0
    def pr(key):
        return market.get(key, {}).get("price") or 0.0
    f = {}
    f["nifty_change"] = chg("nifty")
    f["nifty_change_1w"] = market.get("nifty", {}).get("change_1w") or 0.0
    f["nifty_rsi"] = market.get("nifty", {}).get("rsi") or 50.0
    f["nifty_sma"] = market.get("nifty", {}).get("above_sma20") or 0
    f["nasdaq_change"] = chg("nasdaq")
    f["sp500_change"] = chg("sp500")
    f["nikkei_change"] = chg("nikkei")
    f["hangseng_change"] = chg("hangseng")
    f["usd_inr_change"] = chg("usd_inr")
    f["crude_change"] = chg("crude")

```

```

f["gold_change"]      = chg("gold")
f["us_vix"]           = pr("vix") or 15.0
f["india_vix"]        = pr("india_vix") or 14.0
f["rsi_signal"]       = 1 if f["nifty_rsi"] < 35 else (-1 if f["nifty_rsi"] >
f["trend_signal"]     = 1 if f["nifty_sma"] else -1
f["asia_bull"]        = 1 if (f["nikkei_change"] > 0.5 and f["hangseng_change"
f["asia_bear"]        = 1 if (f["nikkei_change"] < -0.5 and f["hangseng_change"
f["global_bull"]      = 1 if (f["nasdaq_change"] > 0.5 and f["sp500_change"] >
f["global_risk"]      = 1 if (f["us_vix"] > 20 or f["crude_change"] > 2.5) els
f["gold_fear"]        = 1 if (f["gold_change"] > 1.0 and f["us_vix"] > 18) els
f["weekly_trend"]     = 1 if f["nifty_change_1w"] > 1.5 else (-1 if f["nifty_c
return f

def detect_regime(f):
    if f["india_vix"] > 20 or f["us_vix"] > 25:
        return "volatile"
    if f["nifty_rsi"] < 35:
        return "oversold"
    if f["nifty_rsi"] > 65:
        return "overbought"
    if f["nifty_sma"] and f["nifty_change_1w"] > 2.0:
        return "trending"
    return "range"

WEIGHTS = {}
WEIGHTS["nifty_change"]      = 0.18
WEIGHTS["nasdaq_change"]     = 0.12
WEIGHTS["sp500_change"]      = 0.08
WEIGHTS["nikkei_change"]     = 0.08
WEIGHTS["hangseng_change"]   = 0.06
WEIGHTS["usd_inr_change"]    = -0.10
WEIGHTS["crude_change"]      = -0.08
WEIGHTS["global_risk"]       = -0.10
WEIGHTS["global_bull"]       = 0.08
WEIGHTS["asia_bull"]         = 0.06
WEIGHTS["asia_bear"]         = -0.06
WEIGHTS["rsi_signal"]        = 0.08
WEIGHTS["trend_signal"]      = 0.06
WEIGHTS["weekly_trend"]      = 0.05
WEIGHTS["gold_fear"]         = -0.05

def compute_signal(f, regime):
    score = 0.5
    continuous = ["nifty_change", "nasdaq_change", "sp500_change", "nikkei_change", "
    for feat in WEIGHTS:
        val = f.get(feat, 0)
        if feat in continuous:

```

```

        val = max(-3, min(3, val)) / 3.0
        score += WEIGHTS[feat] * val
    if regime == "volatile":
        score = 0.5 + (score - 0.5) * 0.5
    elif regime == "trending":
        score = 0.5 + (score - 0.5) * 1.3
    elif regime == "oversold":
        score = max(score, 0.55)
    elif regime == "overbought":
        score = min(score, 0.45)
    score = max(0.05, min(0.95, score))
    if score > 0.62:
        direction = "bullish"
    elif score < 0.38:
        direction = "bearish"
    else:
        direction = "neutral"
    return {"direction": direction, "probability": round(score, 4), "confidence":

def build_drivers(f):
    drivers = []
    n = f["nifty_change"]
    d1 = {}
    d1["icon"] = "

```

```

d4["tag"] = "neg" if fx > 0.2 else "pos"
d4["label"] = "INR " + ("weakening" if fx > 0.2 else "stable")
d4["detail"] = "USD/INR " + str(round(fx,2)) + "%"
drivers.append(d4)
cr = f["crude_change"]
d5 = {}
d5["icon"] = "🛢️"
d5["tag"] = "neg" if cr > 1 else "pos"
d5["label"] = "Crude " + ("rising" if cr > 1 else "stable")
d5["detail"] = str(round(cr,2)) + "%"
drivers.append(d5)
vix = f["india_vix"]
d6 = {}
d6["icon"] = "⚡" if vix > 18 else "😬"
d6["tag"] = "neg" if vix > 18 else "pos"
d6["label"] = "India VIX " + str(round(vix,1)) + " | US VIX " + str(round(f["
d6["detail"] = "elevated" if vix > 18 else "calm"
drivers.append(d6)
return drivers

def run_backtest():
    try:
        t = yf.Ticker("^NSEI")
        hist = t.history(period="3mo", interval="1d")
        if hist.empty or len(hist) < 30:
            return {"error": "Not enough data"}
        close = hist["Close"].dropna()
        correct = 0
        total = 0
        last_10 = []
        for i in range(20, len(close) - 1):
            current = float(close.iloc[i])
            next_day = float(close.iloc[i + 1])
            actual = "bullish" if next_day > current else "bearish"
            delta = close.diff()
            gain = delta.clip(lower=0).iloc[i-14:i].mean()
            loss = -delta.clip(upper=0).iloc[i-14:i].mean()
            rsi = 50.0
            if loss and loss > 1e-6:
                rsi = float(100 - (100 / (1 + gain / loss)))
            sma20 = float(close.iloc[i-20:i].mean())
            chg = float((current - float(close.iloc[i-1])) / float(close.iloc[i-1]
            chg5 = float((current - float(close.iloc[i-5])) / float(close.iloc[i-5]
            score = 0.5
            score += 0.25 * max(-1, min(1, chg / 3.0))
            score += 0.10 * max(-1, min(1, chg5 / 5.0))
            if rsi < 35:

```

```

        score += 0.10
    elif rsi > 65:
        score -= 0.10
    score += 0.08 if current > sma20 else -0.08
    score = max(0.05, min(0.95, score))
    predicted = "bullish" if score > 0.5 else "bearish"
    if predicted == actual:
        correct += 1
    total += 1
    if len(last_10) < 10:
        row = {}
        row["date"] = hist.index[i].strftime("%Y-%m-%d")
        row["predicted"] = predicted
        row["actual"] = actual
        row["correct"] = predicted == actual
        row["confidence"] = round(score * 100, 1)
        last_10.append(row)
    accuracy = round(correct / total * 100, 1) if total > 0 else 0
    return {"accuracy": accuracy, "total_days": total, "correct": correct, "1
           "summary": "Model correct " + str(accuracy) + "% over 3 months"}
except Exception as e:
    traceback.print_exc()
    return {"error": str(e)}

def get_sparkline():
    try:
        t = yf.Ticker("^NSEI")
        hist = t.history(period="1mo", interval="1d")
        closes = hist["Close"].dropna().tail(20).tolist()
        return [round(float(v), 2) for v in closes]
    except Exception:
        return []

def build_telegram_msg(data, session):
    sig = data.get("signal", {})
    mkt = data.get("market", {})
    drivers = data.get("drivers", [])
    regime = data.get("regime", "range")
    direction = sig.get("direction", "neutral").upper()
    confidence = sig.get("confidence", 0)
    nifty = mkt.get("nifty", {})
    price = nifty.get("price", 0) or 0
    chg = nifty.get("change_pct", 0) or 0
    rsi = nifty.get("rsi", 0) or 0
    india_vix = mkt.get("india_vix", {}).get("price", 0) or 0
    sig_emoji = "🟢" if direction == "BULLISH" else ("🔴" if direction == "BEARISH" else "⚪")
    regime_map = {}

```

```

regime_map["volatile"]    = "HIGH VOL"
regime_map["trending"]   = "TRENDING"
regime_map["oversold"]    = "OVERSOLD"
regime_map["overbought"] = "OVERBOUGHT"
regime_map["range"]      = "RANGE"
regime_text = regime_map.get(regime, regime.upper())
arrow = "up" if chg >= 0 else "down"
session_text = "MORNING" if session == "morning" else "EVENING"
date_str = datetime.now().strftime("%d %b %Y")
driver_lines = ""
for d in drivers[:4]:
    driver_lines += d.get("icon", "") + " " + d.get("label", "") + "
"
msg = "Nifty AI v2.0
"
msg += session_text + " | " + date_str + "
"
msg += "Nifty: " + str(price) + " (" + arrow + " " + str(abs(chg)) + "%)
"
msg += "RSI: " + str(rsi) + " | VIX: " + str(india_vix) + "
"
msg += "Regime: " + regime_text + "
"
msg += "Signal: " + direction + " (" + str(confidence) + "%)
"
msg += "Drivers:
" + driver_lines
msg += "
Dashboard: " + DASHBOARD_URL + "
"
msg += "Not financial advice"
return msg

def send_telegram(message):
    if not TELEGRAM_TOKEN or not CHAT_ID:
        return False, "Missing env vars"
    url = "https://api.telegram.org/bot" + TELEGRAM_TOKEN + "/sendMessage"
    payload = {}
    payload["chat_id"] = CHAT_ID
    payload["text"] = message
    try:
        res = requests.post(url, json=payload, timeout=30)
        return (True, "sent") if res.status_code == 200 else (False, res.text)
    except Exception as e:
        return False, str(e)

```

```

@app.route("/", methods=["GET"])
def index():
    return jsonify({"name": "Nifty AI", "version": "2.0", "status": "running"})

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200

@app.route("/api/signal", methods=["GET"])
def api_signal():
    try:
        market = {}
        for name in TICKERS:
            market[name] = get_cached(TICKERS[name])
        feats = compute_features(market)
        regime = detect_regime(feats)
        signal = compute_signal(feats, regime)
        drivers = build_drivers(feats)
        sparkline = get_sparkline()
        result = {}
        result["status"] = "ok"
        result["timestamp"] = datetime.now().isoformat()
        result["signal"] = signal
        result["regime"] = regime
        result["drivers"] = drivers
        result["sparkline"] = sparkline
        mkt = {}
        mkt["nifty"] = market["nifty"]
        mkt["banknifty"] = market["banknifty"]
        mkt["nasdaq"] = market["nasdaq"]
        mkt["nikkei"] = market["nikkei"]
        mkt["hangseng"] = market["hangseng"]
        mkt["usd_inr"] = market["usd_inr"]
        mkt["crude"] = market["crude"]
        mkt["india_vix"] = market["india_vix"]
        result["market"] = mkt
        return jsonify(result)
    except Exception as e:
        traceback.print_exc()
        return jsonify({"status": "error", "message": str(e)}), 500

@app.route("/api/backtest", methods=["GET"])
def api_backtest():
    try:
        result = run_backtest()
        return jsonify({"status": "ok", "timestamp": datetime.now().isoformat(),

```



```

except Exception as e:
    traceback.print_exc()
    return jsonify({"status": "error", "message": str(e)}), 500

@app.route("/api/telegram", methods=["GET"])
def api_telegram():
    try:
        session = request.args.get("session", "morning")
        market = {}
        for name in TICKERS:
            market[name] = get_cached(TICKERS[name])
        feats = compute_features(market)
        regime = detect_regime(feats)
        signal = compute_signal(feats, regime)
        drivers = build_drivers(feats)
        data = {}
        data["signal"] = signal
        data["regime"] = regime
        data["drivers"] = drivers
        data["market"] = market
        message = build_telegram_msg(data, session)
        ok, res = send_telegram(message)
        if ok:
            return jsonify({"status": "ok", "message": "Sent"})
        return jsonify({"status": "error", "message": res}), 500
    except Exception as e:
        traceback.print_exc()
        return jsonify({"status": "error", "message": str(e)}), 500

@app.route("/api/telegram/test", methods=["GET"])
def api_telegram_test():
    ok, res = send_telegram("Nifty AI v2.0 connected! 15 signals | Asia | Backtes
    if ok:
        return jsonify({"status": "ok"})
    return jsonify({"status": "error", "message": res}), 500

@app.route("/api/data", methods=["GET"])
def api_data():
    market = {}
    for name in TICKERS:
        market[name] = get_cached(TICKERS[name])
    return jsonify({"status": "ok", "timestamp": datetime.now().isoformat(), "mar

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)

```

